Projekt nr 12

Algorytmy i Struktury Danych

Zwiedzanie Muzeów

Maria Eberhardt, Krystian Kaliś

16.12.2025

Spis treści

1 Streszczenie **2**

2 Polecenie zadania **2**

3 Wstęp **2**

4 Algorytm Dijkstry **3**
4.1 Działanie algorytmu 3
4.2 Przykład działania algorytmu 4
4.3 Pseudokod algorutmu 6
4.4 Złożoność algorytmu 8

5 Algorytm wyznaczania pełnej trasy wycieczki **11**

6 Implementacja zadania **12**
6.1 Opis implementacji 12
6.2 Przyjęte rozwiązania techniczne 12
6.3 Struktura aplikacji i wersja wykonywalna 13

7 Testy działania implementacji **14**

8 Wnioski końcowe **20**

9 Podział prac **21**

1 Streszczenie

W pracy przedstawiono rozwiązanie zadania dotyczącego implementacji algorytmu Dijkstry. Pokazano dokładny opis algorytmu, omówienie jego złożoności, pseudokod pokazujący istotę rozwiązanego problemu, przedstawiono implementację w języku Python, przyjęte rozwiązania techniczne oraz opis przeprowadzonych testów.

2 Polecenie zadania

Poniżej przedstawiono treść poruszanego zadania.

Bitowice to miasto o bardzo wielu atrakcjach turystycznych. Jest tam $K <= 20$ bardzo ciekawych muzeów. Sieć komunikacyjna w Bitowicach obejmuje $N <= 50$ przystanków autobusowych, które łączy $M <= 100$ ulic. Przystanek nr 1 to dworzec kolejowy. Każde dwa przystanki łączy co najwyżej jedna linia autobusowa i z każdego przystanku można dotrzeć do wszystkich pozostałych wybierając połączenia autobusowe. Autobusy kursują w obie strony. Znany jest czas przejazdu autobusem pomiędzy przystankami. Przy każdym z K muzeów jest dokładnie jeden przystanek autobusowy.
W najbliższy weekend Bajtazar postanowił przyjechać wczesnym rankiem do Bitowic i zwiedzić wszystkie muzea. Swoją trasę rozpoczyna i kończy na przystanku autobusowym przy dworcu kolejowym. Bajtazar postanowił, że będzie zwiedzał muzea w kolejności rosnącej numerów przystanków, przy których znajdują się te muzea. Niestety, czas ma ograniczony, więc chce tak określić trasę przejazdu autobusami, by łączny czas przejazdu był minimalny.

Wejście:
Należy opcjonalnie wylosować lub wprowadzić z klawiatury następujące dane:

1. *liczbę N przystanków autobusowych,*
2. *liczbę M połączeń między przystankami,*
3. *liczbę $K <= N$ muzeów,*
4. *połączenia autobusowe w postaci ciągu M trójek (x,y,t) : od przystanku o numerze x istnieje bezpośrednie połączenie autobusowe do przystanku o numerze y z czasem przejazdu $t <= 5$.*

Wyjście:
Program podaje minimalny czas przejazdu autobusami na trasie wycieczki Bajtazara.

3 Wstęp

Problem dotyczy znalezienia minimalnego łącznego czasu przejazdu (czyli najkrótszej trasy) pomiędzy przystankami autobusowymi, przy których znajdują się muzea.
Miasto Bitowice posiada:

1. N przystanków autobusowych ponumerowanych od 1 do N,
2. M dwukierunkowych ulic łączących przystanki, z podanym czasem przejazdu t,
3. K przystanków, przy których znajdują się muzea.

Bajtazar rozpoczyna i kończy podróż na przystanku nr 1 (dworcu kolejowym). Każdy przystanek z N jest ponumerowany, odwiedzamy je tak, że przechodzimy od przystanku z najniższym numerem do przystanku o najwyższym numerze. Bajtazar porusza się między przystankami autobusami. Przystanki łączy M ulic, wyrusza z przystanku nr 1 – dworca kolejowego.
Celem jest wyznaczenie minimalnego całkowitego czasu przejazdu autobusami na trasie:

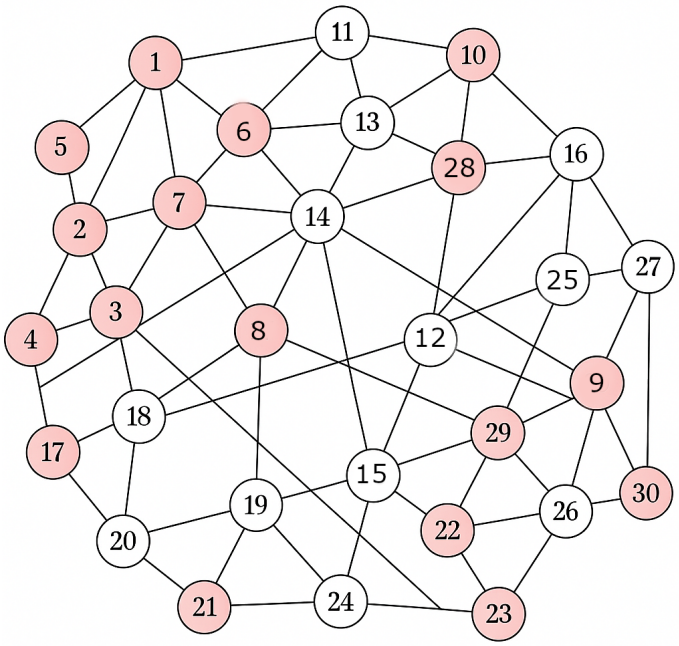
$$1 \to m_1 \to m_2 \to \cdots \to m_K \to 1,$$

gdzie m_i to numery przystanków z muzeami (posortowane rosnąco).

Znane są czasy przejazdu autobusów pomiędzy bezpośrednio połączonymi przystankami. Z każdego przystanku można dotrzeć do każdego innego (graf jest spójny).

Rozważany problem zilustrowano na grafie na rysunku 1.

Zauważmy, że skoro Bajtazar musi odwiedzić przystanki, przy których są muzea w kolejności rosnących numerów, to kolejność obchodzenia tych K przystanków jest ustalona. Minimalizacja długości trasy będzie



Rysunek 1: Przykładowa interpretacja rozważanego problemu dla $N = 30$, $K = 18$ oraz $M = 70$. Mamy 30 możliwych przystanków autobusowych, na których może zatrzymać się Bajtazar, oznaczone są one białymi i różowymi wierzchołkami grafu, każdy z nich jest odpowiednio ponumerowany. Różowe wierzchołki to muzea, jest ich 18, to między nimi Bajtazar będzie chciał podróżować zaczynając od najmniejszego numeru, czyli kolejno będzie chciał odwiedzić wierzchołki oznaczone numerami: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 17, 21, 22, 23, 28, 29, 30. Poruszał się będzie przy tym krawędziami łączącymi przystanki, nie jest to graf skierowany, poruszać się można w obie strony. Każda z krawędzi ma zadaną długość, ponieważ mamy określony czas przejazdy między każdymi z dwóch przystanków. Celem Bajtazara będzie minimalizacja tej trasy. Podróż, czyli obchodzenie takiego grafu zaczynamy od wierzchołka oznaczonego numerem 1.

się zatem opierać na minimalizacji tras pomiędzy poszczególnymi muzeami. Wyznaczenie takich najkrótszych ścieżek pomiędzy wszystkimi istotnymi przystankami w grafie, czyli pomiędzy dworcem oraz przystankami, przy których znajdują się muzea, umożliwia nam algorytm Dijkstry. Ponieważ graf nie jest pełny, a połączenia autobusowe mogą prowadzić tylko między wybranymi przystankami, konieczne jest ustalenie minimalnych czasów przejazdu także dla tras pośrednich. Dijkstra, dzięki temu że działa dla grafów o nieujemnych wagach krawędzi, pozwala efektywnie obliczyć te wartości. Wyniki jego działania wykorzystamy następnie do zbudowania uproszczonego grafu między muzeami i dworcem, w którym każda krawędź odpowiada najkrótszemu możliwemu czasowi przejazdu. W przypadku ograniczenia polegającego na odwiedzaniu muzeów w kolejności rosnących numerów, obliczone odległości posłużą bezpośrednio do zsumowania minimalnych czasów przejazdu pomiędzy kolejnymi punktami trasy.

W trakcie wyznaczenia minimalnego czasu skorzystamy zatem $K + 1$ krotnie z algorytmu Dijkstry, ponieważ na koniec musimy jeszcze znaleźć najkrótszą trasę pomiędzy K-tym muzeum, a dworcem kolejowym znajdującym się na przystanku nr 1. Dzieje się tak pod warunkiem, że na przystanku 1 nie ma muzeum, w innym wypadku wykonujemy algorytm Dijkstry K razy.

4 Algorytm Dijkstry

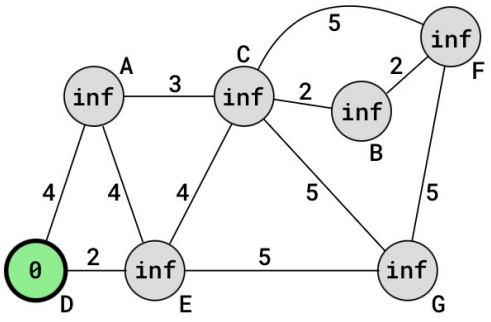
4.1 Działanie algorytmu

Algorytm Dijkstry służy do znajdowania najkrótszych ścieżek w grafie o nieujemnych wagach krawędzi. Działa on w sposób iteracyjny, wybierając w każdym kroku wierzchołek o najmniejszym znanym dotychczas koszcie dojścia i aktualizując odległości do jego sąsiadów.
Proces ten powtarza się, aż dla wszystkich wierzchołków zostanie wyznaczony minimalny dystans od wierzchołka startowego. Dzięki wykorzystaniu kolejki priorytetowej algorytm osiąga złożoność rzędu $O((N + M) \log N)$ [1], co pozwala na szybkie obliczenia nawet dla większych grafów. Przykładowym zastosowaniem algorytmu Dijkstry jest wyznaczanie najkrótszej trasy pomiędzy przystankami w sieci połączeń autobusowych, przy uwzględnieniu rzeczywistych czasów przejazdu pomiędzy poszczególnymi punktami. W niniejszym projekcie wykorzystano go do ustalenia minimalnych czasów przejazdu pomiędzy dworcem a muzeami oraz między samymi muzeami, co

umożliwiło dalszą optymalizację trasy zwiedzania.
W dalszej części projektu zostanie wyprowadzona złożoność oraz implementacja algorytmu.

4.2 Przykład działania algorytmu

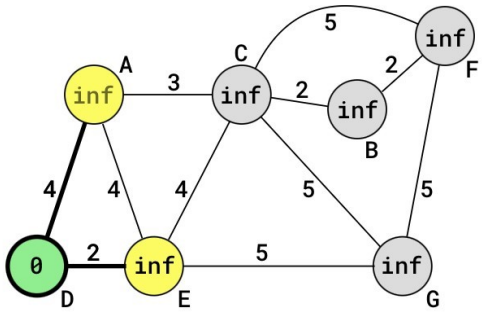
Rozważmy prosty graf składający się siedmiu wierzchołków: A, B, C, D, E, F oraz G. Graf ten przedstawiono na rysunku 2.



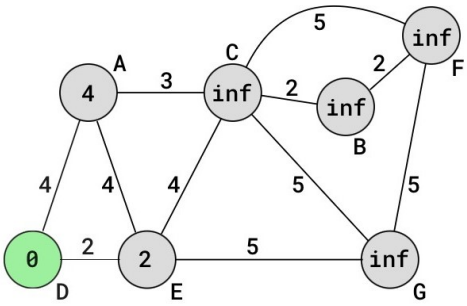
Rysunek 2: Rozważany graf. Algorytm ma nadany punkt startowy D, czas dojścia do niego wynosi 0, wszystkie pozostałe wierzchołki nie były wcześniej odwiedzone, dlatego są opisane umowną nieskończonością inf. [2]

Obok każdej z krawędzi znajduje się waga stanowiąca o długości jaka dzieli wierzchołki na krańcach tej krawędzi, w dalszej części nazywana jest ona jako koszt. Celem jest wyznaczenie najkrótszych ścieżek z wierzchołka D do wszystkich pozostałych.

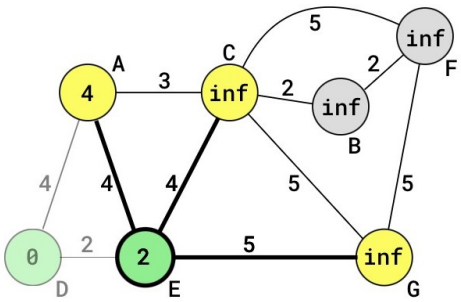
Na rysunkach 3-8 został zilustrowany sposób działania algorytmu Dijkstry podczas rozważania kolejnych iteracji.



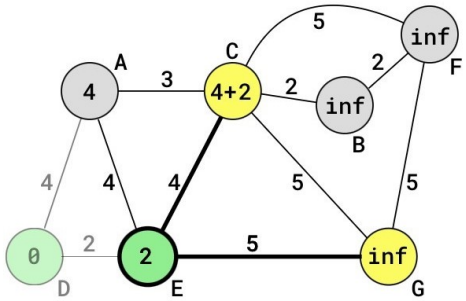
Rysunek 3: Algorytm startujący z wierzchołka D w pierwszym kroku rozważa wierzchołki z nim sąsiadujące. [2]



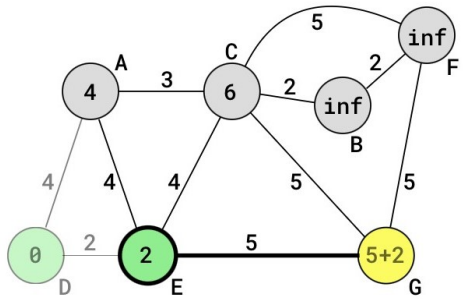
Rysunek 4: Algorytm nadaje rozważanym wierzchołkom jako indeks czas dojścia do nich z wierzchołka D. [2]



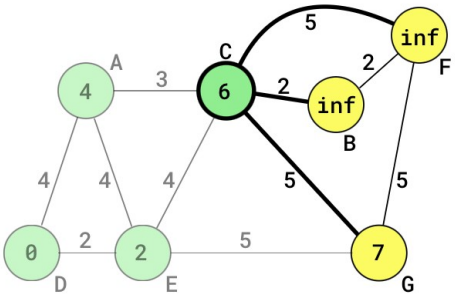
Rysunek 5: Jako kolejny punkt, algorytm wybiera ten wierzchołek, do którego dojdzie mniejszym kosztem (szybciej). Dalej ponownie będziemy rozważać wierzchołki sąsiadujące z naszym nowym wyszczególnionym E. [2]



Rysunek 6: Tym razem liczbami jakie algorytm przyzna wierzchołkom nie będzie po prostu czas dojścia od poprzednika E, ale cały czas dojścia od punktu startowego D. [2]



Rysunek 7: Jak widać, czas dojścia od D do naszego wierzchołka jest liczony jako suma dojścia od D do E i od E do rozważanego punktu (tu G). [2]



Rysunek 8: Algorytm działa tak jak już ustaliliśmy – wybiera wierzchołek do którego dotrze najmniejszym kosztem, przy czym nie będzie wracał do tych wcześniej odwiedzonych, zaznaczonych na zielono. [2]

4.3 Pseudokod algorutmu

W tej sekcji przedstawiono pseudokod dla algorytmu Dijkistry [3].

Algorithm 1 Algorytm Dijkstry

```
begin
for all v ∈ V do
    D[v].dist := l(s, v);
end for
S := {s};
while S ≠ V do
    {wyznaczenie z V − S wierzchołka najbliższego s}
    mindist := +∞;
    for all w ∈ V − S do
        if D[w].dist < mindist then
            v := w;
            mindist := D[w].dist;
        end if
    end for
    S := S ∪ {v};
    for all w ∈ V − S do
        if D[w].dist > mindist + l(v, w) then
            D[w].dist := mindist + l(v, w);
            D[w].prev := v;
        end if
    end for
end while
end
```

Opis elementów algorytmu Dijkstry:

- V – zbiór wszystkich wierzchołków (przystanków) w grafie.
- E – zbiór wszystkich krawędzi (połączeń autobusowych) w grafie.
- s – wierzchołek początkowy (w naszym przypadku przystanek nr 1 – dworzec kolejowy).
- $D[v].dist$ – aktualnie znana najmniejsza odległość (czas przejazdu) z wierzchołka początkowego s do wierzchołka v.
- $D[v].prev$ – poprzednik wierzchołka v na najkrótszej znalezionej ścieżce z s do v.
- $l(u, v)$ – waga krawędzi (czas przejazdu) pomiędzy wierzchołkami u i v.
- S – zbiór wierzchołków, dla których ostatecznie wyznaczono najkrótsze ścieżki z wierzchołka s (wierzchołki już przetworzone).
- $V - S$ – zbiór wierzchołków, dla których najkrótsza ścieżka nie została jeszcze ustalona.

- *mindist* – tymczasowa zmienna przechowująca najmniejszą odległość spośród wierzchołków nieprzetworzonych.
- v – wierzchołek aktualnie przetwarzany (ten o najmniejszej wartości $D[v].dist$ spośród $V - S$).
- w – sąsiad wierzchołka v.

W naszej implementacji posłużymy się uproszczoną wersją algorytmu Dijkistry, ponieważ przy zadanym problemie musimy szukać jedynie optymalnych tras do konkretnych przystanków, a nie do wszystkich. Modyfikacja będzie polegać na tym, że nasz algorytm zatrzyma się w momencie osiągnięcia konkretnego, poszukiwanego punktu [4]. Zmodyfikowany pseudokod algorytmu przedstawiono poniżej. Algorytm jako argumenty będzie przyjmował postać rozważanego grafu, czyli planu Bitowic oraz dwa punkty, między którymi będziemy wyznaczać najkrótszą ścieżkę: punkt startowy s oraz punkt docelowy t.

Algorithm 2 Uproszczony algorytm Dijkstry dla jednego celu

```
1:  begin
2:  for all v ∈ V do
3:      D[v].dist := l(s, v);
4:  end for
5:  S := {s};
6:  while S ≠ V do
7:      {wyznaczenie z V − S wierzchołka najbliższego s}
8:      mindist := +∞;
9:      for all w ∈ V − S do
10:         if D[w].dist < mindist then
11:             v := w;
12:             mindist := D[w].dist;
13:         end if
14:     end for
15:     S := S ∪ {v};
16:     if v = t then
17:         break;
18:     end if
19:     for all w ∈ V − S do
20:         if D[w].dist > mindist + l(v, w) then
21:             D[w].dist := mindist + l(v, w);
22:             D[w].prev := v;
23:         end if
24:     end for
25: end while
26: Path := t;
27: curr := D[t].prev;
28: while curr ≠ s do
29:     Path := Path ∪ curr;
30:     curr := D[curr].prev;
31: end while;
32: Path := Path ∪ s;
33: for i := 0 to ⌊length(Path)/2⌋ − 1 do
34:     temp := Path[i];
35:     Path[i] := Path[k − 1 − i];
36:     Path[k − 1 − i] := temp;
37: end for
38: return D[t].dist, Path
39: end
```

Zmiany wprowadzone w tym kodzie względem standardowego algorytmu Dijkstry (Algorytm 1) mają na celu przystosowanie go do wyszukiwania drogi do konkretnego celu t oraz jawnego zwrócenia trasy. Modyfikacje obejmują trzy kluczowe sekcje:

- **Wczesne zatrzymanie (wiersze 16–18):** Dodano instrukcję warunkową sprawdzającą, czy wierzchołek v, który właśnie został wybrany jako najbliższy (dodany do zbioru S), jest poszukiwanym celem t. Jeśli

tak, algorytm natychmiast przerywa pętlę `while`. Pozwala to zaoszczędzić czas obliczeniowy, unikając zbędnego przetwarzania pozostałej części grafu.

- **Rekonstrukcja ścieżki (wiersze 26–32):** Po znalezieniu celu następuje proces odtwarzania trasy, czyli *backtracking*. Wykorzystując tablicę poprzedników `prev`, algorytm cofa się iteracyjnie od wierzchołka docelowego t aż do startowego s, zapisując kolejne kroki na liście `Path`.

- **Odwrócenie kolejności (wiersze 33–37):** Ponieważ ścieżka jest odtwarzana od końca (od t do s), konieczne jest odwrócenie kolejności elementów tablicy `Path`, aby uzyskać poprawny kierunek trasy (od s do t). Algorytm wykonuje to w miejscu (*in-place*) poprzez zamianę elementów symetrycznych.

Ostatecznie funkcja kończy działanie w wierszu 38, zwracając parę wartości: minimalny koszt dotarcia do celu (`D[t].dist`) oraz uporządkowaną listę wierzchołków (`Path`).

4.4 Złożoność algorytmu

Niech $G = (V, E)$ będzie spójnym digrafem o nieujemnych wagach krawędzi, gdzie $|V| = n$ oraz $|E| = m$. Niech $s \in V$ będzie wierzchołkiem startowym (dworzec kolejowy), a $t \in V$ – wierzchołkiem docelowym (przystanek przy wybranym muzeum). Analizowany algorytm Dijkstry (algorytm 1) został zmodyfikowany tak, aby zakończyć swoje działanie w momencie odnalezienia najkrótszej ścieżki do t, czyli po spełnieniu warunku $v = t$, gdzie v jest rozważanym wierzchołkiem digrafu.

Niech $|S| = i$ oznacza liczbę wierzchołków już odwiedzonych po i iteracjach pętli głównej, a i^* oznacza liczbę iteracji do momentu osiągnięcia wierzchołka t. Wprowadzamy oznaczenie z gwiazdką, ponieważ nie wiemy w którym kroku dojdziemy do wierzchołka t. W analizie przyjmujemy, że koszt pojedynczej operacji (porównania, przypisania) jest równy pewnej stałej C.

Oczywiście, nie zachodzi $|S| = |Path|$, ponieważ S jest zbiorem wszystkich wierzchołków, które algorytm musiał odwiedzić i zatwierdzić wyznaczyć odległość, zanim dotarł do celu. To jest nasz obszar poszukiwań. Zbiór $Path$ to liczba wierzchołków na ostatecznej ścieżce, a zatem nasz wynik. Z reguły więc zazwyczaj $|S|$ jest znacznie większe niż $|Path|$. Możemy zatem zauważyć, że pętla główna (wiersze 6-25) kręci się tyle razy, ile wierzchołków wrzucimy do S. Zatem koszt algorytmu zależy od wielkości i^*, a nie od $|Path|$. Wielkość ta zaś jest zależna od wierzchołka startowego s i końcowego t Wzór na złożoność algorytmu wyprowadzimy biorąc pod uwagę najbardziej kosztowną część algorytmu, czyli pętlę główną. Wspominając o pozostałych częściach pseudokodu (linie 26-38), przy założeniu $|Path| = k$ mamy:

- Linie 26–32 – Rekonstrukcja ścieżki. Pętla while cofa się od celu t do startu s po referencjach prev. Wykonuje się k razy, czyli mamy $O(k)$.

- Linie 33–37 – Odwracanie tablicy. Pętla for idzie od indeksu 0 do połowy długości tablicy (length(Path)/2), zamieniając elementy miejscami. Czyli mamy złożoność: $O(k/2)$, co w notacji asymptotycznej upraszcza się do $O(k)$.

- Linia 38 – Zwracanie wyniku. Złożoność: $O(1)$.

Wielkości te są znikome w porównaniu z główną pętlą algorytmu, którą teraz rozważymy.

Przeanalizujemy krok po kroku działanie naszego algorytmu uwzględniając wkład do złożoności od poszczególnych linii pseudokodu w pętli głównej.

- Linie (2–4): inicjalizacja odległości dla wszystkich wierzchołków $v \in V$ wymaga n operacji. C_1 jest stałym kosztem, który odpowiada przypisaniu wag do zmiennej przechowującej odległości (linijka 3).

$$T_1 = C_1 \cdot n$$

- Linie (9–14): w każdej iteracji pętli głównej wyszukiwany jest wierzchołek $v \in V - S$ o najmniejszej wartości $D[v].dist$. Liczba porównań w i-tej iteracji wynosi $n - i$, zatem koszt:

$$T_2(i) = C_2 \cdot (n - i),$$

  gdzie C_2 to stały koszt funkcji if wewnątrz pętli.

- Linie (19–24): relaksacja krawędzi dla wierzchołków $w \in V - S$ wymaga sprawdzenia wszystkich $n - i - 1$ elementów. Operacje we wnętrzu tej pętli mają stały koszt wynoszący C_3. Zatem całkowity koszt pętli for to:

$$T_3(i) = C_3 \cdot (n - i - 1)$$

Łączny koszt i-tej iteracji pętli głównej (linie 6-25) wynosi więc:

$$T(i) = T_2(i) + T_3(i) = C_2(n-i) + C_3(n-i-1) + C_4,$$

gdzie C_4 jest stałym kosztem wynikającym z instrukcji warunkowej w linijkach 16-18 oraz nadpisania zmiennej w linijce 15.
Algorytm wykonuje i^* iteracji pętli głównej, aż do znalezienia wierzchołka docelowego t. Całkowity koszt czasowy można zatem zapisać jako:

$$T(n,t) = T_1 + \sum_{i=1}^{i^*(t)} T(i),$$

koszt ten jest zależny od ilości wierzchołków grafu n oraz od wyróżnionego wierzchołka t, do którego wykonujemy obliczenia.

$$T(n,t) = C_1 n + \sum_{i=1}^{i^*(t)} [C_2(n-i) + C_3(n-i-1) + C_4]$$

$$T(n,t) = C_1 n + (C_2 + C_3) \sum_{i=1}^{i^*(t)} (n-i) + (C_4 - C_3) i^*(t),$$

czyli:

$$T(n,t) = C_1 n + (C_2 + C_3) \left(\frac{n - 1 + n - i^*(t)}{2} \cdot i^*(t) \right) + (C_4 - C_3) i^*(t)$$

Zatem po uproszczeniu otrzymujemy wzór na koszt czasowy algorytmu dany równaniem:

$$\boxed{T(n,t) = C_1 n + (C_2 + C_3) \left(i^*(t) n - \frac{i^*(t)(i^*(t)+1)}{2} \right) + (C_4 - C_3) i^*(t)} \tag{1}$$

Oszacujemy teraz tę zależność na koszt czasowy algorytmu z góry i z dołu aby wyznaczyć złożoność czasową.
W tym celu na początku uprościmy stałe i notację. Aby zwiększyć czytelność przekształceń, zastąpimy grupy stałych współczynników pojedynczymi symbolami, przy założeniu, że stałe kosztów operacji są dodatnie:

$$A = C_2 + C_3 \quad \text{(koszt operacji wewnątrz pętli)}$$
$$B = C_4 - C_3 \quad \text{(koszt instrukcji warunkowych i przypisań)}$$
$$C = C_1 \quad \text{(koszt inicjalizacji)}$$

Po podstawieniu, wzór 1 przyjmuje postać:

$$T(n,t) = C \cdot n + A \underbrace{\left(i^*(t) n - \frac{i^*(t)(i^*(t)+1)}{2} \right)}_{\text{Główny składnik } S} + B \cdot i^*(t)$$

Skupmy się na wyrażeniu w nawiasie, które dominuje w rzędzie złożoności. Sprowadźmy je do wspólnego mianownika i wyłączmy czynnik przed nawias:

$$S = i^*(t) n - \frac{i^2 + i^*(t)}{2}$$

$$S = \frac{2 i^*(t) n - (i^*(t))^2 - i^*(t)}{2}$$

$$S = \frac{i^*(t)}{2} (2n - i^*(t) - 1)$$

Otrzymana postać $S = \frac{i^*(t)}{2}(2n - i^*(t) - 1)$ jest kluczowa dla dalszych oszacowań.
Dokonujemy kolejno oszacowań:

1. **Oszacowanie górne** (Big O)
   Szukamy stałej K, dla której $T(n,t) \leq K \cdot (i^*(t) n)$. Rozważmy składnik S. Ponieważ $i^*(t) \geq 1$ oraz $n \geq 1$, to odejmując wartości dodatnie $(i^*(t) + 1)$ od $2n$, otrzymujemy:

   $$(2n - i^*(t) - 1) < 2n$$

   Stąd wynika oszacowanie dla S:

   $$S < \frac{i^*(t)}{2} \cdot (2n) = i^*(t) n$$

   Analizując pozostałe składniki:

- $C \cdot n \leq C(i^*(t)n)$ (ponieważ $i^*(t) \geq 1$)
- $B \cdot i^*(t) \leq B(i^*(t)n)$ (ponieważ $n \geq 1$)

Sumując te nierówności dla całego wzoru:

$$T(n,t) < C(i^*(t)n) + A(i^*(t)n) + B(i^*(t)n) = (C + A + B) \cdot (i^*(t)n)$$

Istnieje zatem stała $K = C + A + B$, co dowodzi, że:

$$T(n,t) = O(i^*(t) \cdot n)$$

2. **Oszacowanie dolne** (Big Ω)
Szukamy stałej $L > 0$, dla której $T(n,t) \geq L \cdot (i^*(t)n)$. Musimy oszacować wyrażenie $(2n - i^*(t) - 1)$ z dołu w najgorszym możliwym przypadku. Najmniejszą wartość nawias przyjmie, gdy $i^*(t)$ jest maksymalne. Wiemy, że $i^*(t) < n$, zatem:

$$2n - i^*(t) - 1 > 2n - n - 1 = n - 1$$

Dla $n \geq 2$ zachodzi $n - 1 \geq \frac{n}{2}$. Podstawiając to do wzoru na S:

$$S \geq \frac{i^*(t)}{2} \cdot \frac{n}{2} = \frac{1}{4} i^*(t)n$$

Ponieważ $C \cdot n > 0$, możemy pominąć mniejsze składniki przy szacowaniu z dołu, skupiając się na składniku dominującym:

$$T(n,t) > A \cdot S \geq A \cdot \frac{1}{4} i^*(t)n$$

Istnieje zatem stała $L = \frac{A}{4}$, co dowodzi, że:

$$T(n,t) = \Omega(i^*(t) \cdot n)$$

Ponieważ funkcja kosztu $T(n,t)$ jest ograniczona z obu stron przez funkcję liniową względem iloczynu $i^*(t) \cdot n$, czyli:

$$\Omega(i^*(t) \cdot n) \leq T(n,t) \leq O(i^*(t) \cdot n),$$

to ostatecznie złożoność asymptotyczna algorytmu Dijistry z warunkiem zatrzymania wynosi:

$$\boxed{T(n,t) = \Theta(i^*(t) \cdot n)} \tag{2}$$

Dla pełnej wersji algorytmu, czyli w najgorszym przypadku, liczba iteracji wyniesie $i^*(t) = n - 1$, zatem:

$$T(n,t) = C_1 n + (C_2 + C_3)\left((n-1)n - \frac{(n-1)n}{2}\right) + (C_4 - C_3)(n-1),$$

co zgodnie ze wzorem (2) oraz biorąc pod uwagę to, że interesuje nas składnik najszybciej rosnący daje nam:

$$\boxed{T(n,t) = \Theta(n^2).} \tag{3}$$

Mamy więc w tym wypadku zbieżność kwadratową algorytmu. Taka złożoność nas interesuje, bo złożoność obliczeniowa dotyczy najgorszego przypadku.
Możemy także zauważyć, że w najlepszym przypadku, czyli gdy od razu po inicjalizacji poszukiwania wierzchołka, w którym jest muzeum, znajdziemy się w nim, to $i^* = 1$. Złożoność jest zatem liniowa:

$$T(n,t) = \Theta(n).$$

Podsumowując, zmodyfikowany algorytm Dijkstry w przypadku osiągnięcia wierzchołka docelowego t w pewnym kroku zachowuje złożoność proporcjonalną do liczby wykonanych iteracji i^*. Dzieje się tak dlatego, że dla algorytmu z warunkiem break nie można podać jednej, ogólnej złożoności $\Theta(f(n))$ dla wszystkich przypadków, ponieważ czas działania zmienia się zbyt drastycznie w zależności od danych wejściowych, czyli od położenia wierzchołka t. Zatem w przeciwieństwie do klasycznej wersji algorytmu, która zawsze wykonuje pracę rzędu $\Theta(n^2)$, wersja zmodyfikowana w przypadkach optymistycznych, czyli jeśli cel znajduje się blisko źródła redukuje czas działania do liniowego $\Theta(n)$.

5 Algorytm wyznaczania pełnej trasy wycieczki

Głównym celem projektu jest znalezienie trasy o minimalnym czasie, która rozpoczyna się na dworcu kolejowym (węzeł oznaczony numerem 1), odwiedza zbiór K muzeów w kolejności rosnących numerów przystanków, a następnie kończy się powrotem na dworzec.
Kolejność odwiedzania punktów jest z góry narzucona, posortowana rosnąco według indeksów wierzchołków, zatem, jak już ustaliliśmy, problem dzieli się na serię $K + 1$ niezależnych problemów znalezienia najkrótszej ścieżki między dwoma ustalonymi punktami. Do tego służy nam opisany w sekcji 4 algorytm Dijkstry zwracający odległość między dwoma punktami – punktem startowym s, w którym zaczynamy trasę oraz punktem docelowym t. Zwracana jest także ścieżka między nimi.

Poniżej został przedstawiony algorytm pozwalający na rowziązanie rozważanego w poleceniu problemu.

Algorithm 3 Algorytm wyznaczania pełnej trasy wycieczki

Require: Graf $G(V, E)$, wierzchołek startowy s, zbiór muzeów M

1: **begin**
2: $TotalTime := 0$;
3: $FullPath := \{s\}$;
4: $curr := s$;
5: $M := \textsc{Sort}(M)$
6: **for all** $museum \in M$ **do**
7: **if** $curr = museum$ **then**
8: **continue**;
9: **end if**
10: $(dist, segment) := \text{DijkstraStop}(G, curr, museum)$;
11: $TotalTime := TotalTime + dist$;
12: $FullPath := FullPath \cup (segment \setminus \{first(segment)\})$;
13: $curr := museum$;
14: **end for**;
15: **if** $curr \neq s$ **then**
16: $(dist, segment) := \text{DijkstraStop}(G, curr, s)$;
17: $TotalTime := TotalTime + dist$;
18: $FullPath := FullPath \cup (segment \setminus \{first(segment)\})$;
19: **end if**;
20: **return** $TotalTime, FullPath$;
21: **end**;

Algorytm ten jako argumenty przyjmuje graf $G(V, E)$, wierzchołek, w którym rozpoczynany jest algorytm s oraz zbiór muzeów M. Zwraca on całkowity czas obejścia wszystkich muzeów wraz z powrotem do dworca kolejowego oraz całkowitą ścieżkę odwiedzanych po kolei przez Bajtazara wierzchołków.
Działanie algorytmu można podzielić na cztery główne fazy:

1. Wiersze 1–4: algorytm ustawia licznik całkowitego czasu podróży ($TotalTime$) na 0 oraz inicjuje listę wynikową ($FullPath$), umieszczając w niej wierzchołek startowy s (dworzec). Zmienna pomocnicza $curr$, wskazująca aktualne położenie Bajtazara, również ustawiana jest na wartość s.

2. Wiersz 5: zbiór wejściowy M zostaje posortowany rosnąco. Dzięki temu iteracja w pętli głównej będzie przebiegać w wymaganym porządku logicznym.

3. Wiersze 6–14: algorytm iteruje po posortowanej liście muzeów. W kolejnych krokach pętli:

   - Sprawdzamy, czy aktualny wierzchołek ($curr$) nie jest tożsamy z celem ($museum$). Sytuacja taka może wystąpić, gdy np. startujemy z dworca, a dworzec znajduje się na liście muzeów do odwiedzenia. W takim przypadku algorytm pomija iterację (continue), koszt dojścia wynosi 0.
   - Wyznaczamy odcinek trasy. Wywoływany jest Algorytm 2, który oblicza najkrótszą ścieżkę z punktu $curr$ do bieżącego $museum$. Zwraca on długość tego odcinka ($dist$) oraz listę wierzchołków ($segment$).
   - Koszt odcinka jest dodawany do sumy całkowitej $TotalTime$.
   - Nowy segment trasy jest dołączany do głównej ścieżki $FullPath$. Mamy fragment $segment$ $first(segment)$. Sprawia on, że do listy nie jest dodawany wierzchołek, na którym poprzednio ta lista się zakończyła.
   - Zmienna $curr$ przyjmuje wartość odwiedzonego muzeum, stając się punktem startowym dla kolejnej iteracji.

4. Wiersze 15-19: Po odwiedzeniu wszystkich muzeów algorytm sprawdza, czy ostatni odwiedzony punkt jest różny od punktu startowego s. Jeśli tak, następuje ostatnie wywołanie funkcji DijkstraStop dla pary ($curr$, s), a wynikowy czas i ścieżka są dopisywane do rezultatów końcowych analogicznie jak w pętli głównej.

Złożoność całkowita algorytmu wyznaczania pełnej trasy jest sumą kosztów każdego z etapów algorytmu. Jeśli N oznacza liczbę przystanków, a K liczbę muzeów, to mamy:

1. **Sortowanie:** Posortowanie K muzeów wymaga czasu $\Theta(K \log K)$.
2. **Główna pętla:** Algorytm wykonuje K iteracji. W każdej z nich uruchamiany jest Algorytm 2. Pesymistyczna złożoność pojedynczego uruchomienia tego algorytmu wynosi $\Theta(N^2)$. Zatem koszt całej pętli to w najbardziej perymistycznym przypadku $\Theta(K \cdot N^2)$.
3. **Powrót:** Ostatnie wywołanie algorytmu Dijkstry to dodatkowy koszt $\Theta(N^2)$ w najgorszym przypadku.

Całkowitą złożoność pesymistyczną można zapisać więc wzorem:

$$T(N, K) = \Theta(K \log K) + \Theta(K \cdot N^2) + \Theta(N^2)$$

Ponieważ $K \leq N$, składnikiem dominującym jest koszt wielokrotnego uruchomienia algorytmu Dijkstry. Ostatecznie złożoność wynosi:

$$\boxed{T(N, K) = \Theta(K \cdot N^2)} \tag{4}$$

Warto zauważyć, że dla danych wejściowych ograniczonych w treści zadania ($N \leq 50, K \leq 20$), liczba operacji w najgorszym przypadku wyniesie około $20 \cdot 50^2 = 50\,000$, co dla współczesnych procesorów jest wartością pomijalnie małą i gwarantuje natychmiastowe otrzymanie wyniku. Warto również dodać, że dzięki zastosowaniu mechanizmu *early exit* w Algorytmie 2, rzeczywisty czas wykonania będzie zazwyczaj znacznie krótszy niż to pesymistyczne oszacowanie.

6 Implementacja zadania

6.1 Opis implementacji

W implementacji wykorzystano język `Python` oraz środowisko `Jupyter Notebook`, co umożliwiło przejrzyste prezentowanie kolejnych etapów obliczeń. Zaimplementowano funkcję `dijkstra_stop`, odpowiedzialną za wyznaczanie najkrótszych czasów przejazdu pomiędzy wskazanymi przystankami. Funkcja ta stanowi podstawę budowy pełnej trasy zwiedzania, poprzez sukcesywne obliczanie kosztów przejazdu pomiędzy kolejnymi muzeami. Do celów testowych generowane są zarówno losowe grafy o zadanych parametrach, jak i przykłady konstrukcyjne, co pozwala na ocenę poprawności działania programu w różnych konfiguracjach wejściowych. Całość kodu została uporządkowana w sposób ułatwiający analizę poszczególnych modułów oraz ewentualne rozszerzenia.

6.2 Przyjęte rozwiązania techniczne

W implementacji przygotowano zestaw funkcji tworzących spójny moduł obliczeniowy, umożliwiający zarówno generowanie danych wejściowych, jak i pełną symulację zadanej trasy zwiedzania muzeów oraz jej wizualizację.

Zaimplementowane zostały następujące funkcje:

1. `dijkstra_stop` – podstawowy element programu, to funkcja której zadaniem jest obliczanie najkrótszych czasów przejazdu pomiędzy dwoma zadanymi przystankami. Funkcja ta operuje na grafie skierowanym lub nieskierowanym zapisanym w formacie słownika list sąsiedztwa i implementuje klasyczny algorytm Dijkstry. Zanim została włączona do głównego modułu projektu, poddano ją testom na niewielkich, ręcznie skonstruowanych grafach oraz na losowo generowanych przykładach, co umożliwiło ocenę jej poprawności dla różnorodnych struktur wejściowych. Funkcja ta jest implementacją algorytmu 2.
2. `solve_museum_tour` – funkcja opisana przez algorytm 3. To właściwa symulacja zadania, polegająca na wyznaczeniu minimalnego łącznego czasu podróży Bajtazara. Zgodnie z założeniami problemu muzea odwiedzane są w kolejności rosnących numerów przystanków, przy których się znajdują. Funkcja ta iteruje więc po ustalonej liście muzeów i dla każdej kolejnej pary wywołuje funkcję `dijkstra_stop`, obliczając minimalny czas przejazdu między aktualną lokalizacją a następnym muzeum. Wyniki cząstkowe są sumowane, tworząc rosnącą wartość łącznego czasu podróży. Po odwiedzeniu ostatniego muzeum obliczany jest dodatkowo czas powrotu do punktu startowego, co stanowi ostatni etap symulacji.

3. `validate_parameters` – funkcja zwracająca *error* dla nieprawidłowych wartości parametrów `N`, `M` oraz `K`, niezgodnych z treścią problemu.

4. `generate_graph` – funkcja do generowania losowego grafu nieskierowanego o zadanej liczbie węzłów i krawędzi. W razie, gdyby losowe ziarno prowadziło do powstania grafu niespójnego, funkcja automatycznie uzupełnia strukturę o dodatkowe krawędzie, tak aby z całkowitą pewnością otrzymać graf spójny. Proces ten gwarantuje, że z każdego przystanku istnieje możliwość dotarcia do pozostałych, co stanowi fundamentalny warunek poprawnego działania algorytmu Dijkstry i symulacji całej trasy. Po skonstruowaniu struktury grafowej każdej krawędzi przypisywana jest losowa waga od 1 do 5, interpretowana jako czas przejazdu autobusem pomiędzy wybranymi przystankami. Zakres wag odpowiada parametrom przyjętym w treści zadania i pozwala odzwierciedlić realistyczne różnice czasowe pomiędzy połączeniami komunikacyjnymi.

5. `visualize_museum_graph` – funkcja, której zadaniem jest przedstawienie wygenerowanego grafu w postaci graficznej, na podstawie struktury zapisanej w formacie słownika. Reprezentacja taka, zawierająca dla każdego wierzchołka listę par postaci `(sąsiad, waga)`, okazuje się szczególnie dogodna zarówno do generowania danych testowych, jak i do późniejszego ich przetwarzania. Funkcja wizualizacyjna służy nie tylko do estetycznego przedstawienia wyników, lecz także do oceny poprawności wygenerowanego grafu – między innymi jego spójności, równomiernego rozłożenia krawędzi oraz prawidłowości nadanych wag. Z perspektywy użytkowej stanowi ona narzędzie wspierające analizę, umożliwiając natychmiastową kontrolę struktury scenariusza wygenerowanego przed właściwą symulacją.

6. `simulation` – to funkcja integrująca wszystkie wcześniejsze moduły w jeden spójny proces obliczeniowy. Wywołuje wszystkie wymienione powyżej funkcje aby rozwiązać problem dla wczytanych parametrów `N`, `M` oraz `K`, oznaczających odpowiednio liczbę wierzchołków, liczbę krawędzi oraz liczbę muzeów przewidzianych w grafie.

7. `simulation` -– funkcja integrująca wszystkie wcześniejsze moduły w jeden spójny proces obliczeniowy. Obsługuje ona dwa opcjonalne parametry: manual_graph oraz manual_museums. Wprowadzono w niej logikę warunkową, która sprawdza obecność tych argumentów:

   - Jeśli parametry te zostaną przekazane (co ma miejsce w przypadku wybrania przez użytkownika trybu ręcznego), funkcja pomija etap losowego generowania struktury i wykorzystuje graf oraz listę muzeów precyzyjnie zdefiniowaną przez użytkownika.
   - W przypadku braku tych argumentów (tryb automatyczny lub półautomatyczny), funkcja samodzielnie wywołuje generate_graph oraz losuje lokalizacje muzeów.

   Dzięki takiemu podejściu funkcja simulation obsługuje symulacje losowe, jak i przykłady wprowadzane z klawiatury.

8. `pobierz_liczbe` – funkcja pomocnicza odpowiadająca za bezpieczną interakcję z użytkownikiem podczas ręcznego wprowadzania danych . Jej zadaniem jest wymuszenie podania wartości liczbowej całkowitej (int) oraz weryfikacja, czy mieści się ona w zadanym przedziale (min_val, max_val). Funkcja wykorzystuje mechanizm obsługi wyjątków (try-except), aby zapobiec awarii programu w przypadku wprowadzenia przez użytkownika znaków niebędących liczbami, wyświetlając odpowiedni komunikat błędu i prosząc o ponowne wprowadzenie danych.

9. `main` – funkcja stanowiąca główny punkt wejścia do programu. Implementuje ona pętlę główną aplikacji konsolowej, wyświetla menu użytkownika oraz zarządza przepływem sterowania w zależności od wybranej opcji. Funkcja ta integruje wszystkie wcześniej opisane moduły, pozwalając na wybór trybu działania (ręczny, półautomatyczny, w pełni losowy) oraz zapewnia bezpieczne zakończenie działania programu.

6.3 Struktura aplikacji i wersja wykonywalna

W celu ułatwienia obsługi programu przez użytkownika końcowego, kod źródłowy został obudowany interaktywnym interfejsem tekstowym. Struktura aplikacji opiera się na nieskończonej pętli while, która prezentuje menu główne i oczekuje na decyzję użytkownika, co pozwala na wielokrotne przeprowadzanie symulacji bez konieczności ponownego uruchamiania programu. Zaimplementowano cztery główne tryby działania aplikacji:

1. **Tryb ręczny** – Użytkownik samodzielnie definiuje liczbę przystanków (N), połączeń (M) i muzeów (K), a następnie ręcznie wprowadza każdą krawędź grafu (pary wierzchołków i wagi) oraz lokalizacje muzeów. Tryb ten zawiera pełną walidację spójności danych wejściowych.

2. **Tryb półautomatyczny** – Użytkownik podaje jedynie parametry N, M, K, natomiast struktura grafu i rozmieszczenie muzeów są generowane losowo przez algorytm.

3. **Tryb automatyczny** – Program losuje wszystkie parametry oraz strukturę grafu, przechodząc następnie do symulacji i prezentacji wyników.

4. **Wyjście** – Bezpieczne zakończenie pracy aplikacji.

Zwieńczeniem prac implementacyjnych było przygotowanie wersji wykonywalnej projektu (plik .exe). Proces ten zrealizowano przy użyciu narzędzia PyInstaller, co pozwoliło na spakowanie interpretera języka Python oraz wszystkich niezbędnych bibliotek (networkx, matplotlib) do jednego pliku. Dzięki temu aplikacja może być uruchamiana na komputerach z systemem Windows bez konieczności instalowania środowiska programistycznego czy dodatkowych pakietów, co znacząco zwiększa jej przenośność i użyteczność.

Wygląd interfejsu użytkownika po uruchomieniu programu przedstawiono na rysunku 9.

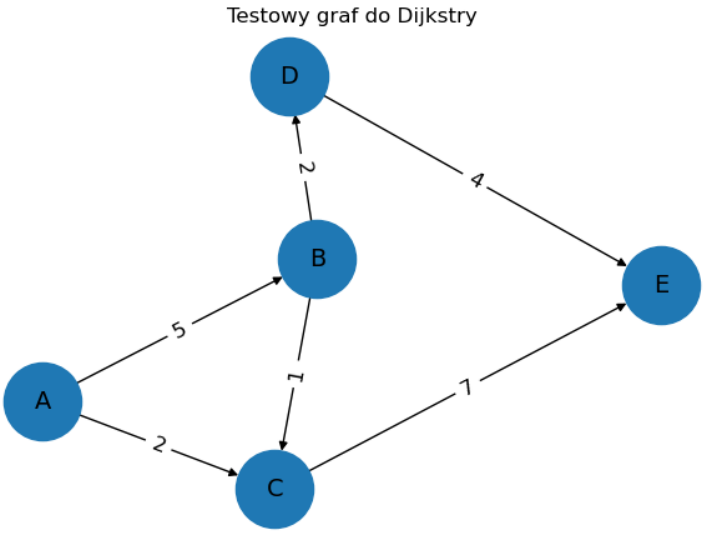
```
Projekt 12: Zwiedzanie Muzeów (Bitowice)
------------------------------------------------------------
Tematyka projektu
Program pomaga Bajtazarowi zaplanować najszybszą trasę wycieczki po muzeach.
Zasady:
1. Początek i koniec trasy są zawsze na Dworcu autobusowym - przystanku nr 1.
2. Bajtazar musi odwiedzić K muzeów rozmieszczonych w mieście.
3. Muzea są zwiedzane w kolejności rosnącej
numerów przystanków, np. najpierw przystanek 3, potem 7, potem 12.
4. Program oblicza minimalny czas przejazdu, sumując najkrótsze
ścieżki między kolejnymi punktami harmonogramu.
5. Mamy łącznie N ponumerowanych przystanków autobusowych oraz M ścieżek łączących je.
------------------------------------------------------------

1) Wprowadź dane całkowicie ręcznie (graf i muzea)
2) Wprowadź parametry (N, M, K) ręcznie, graf i muzea losowe
3) Wygeneruj wszystko losowo
4) Wyjdź z programu
Wprowadź z klawiatury jedną z opcji 1-4:
```

Rysunek 9: Menu główne interfejsu konsolowego aplikacji.

7 Testy działania implementacji

W tej części zbadamy działanie zaimplementowanych funkcji. Testy zostały rozpoczęte od wywołania funkcji `dijkstra_stop` na małym grafie, aby sprawdzić poprawność działania tej funkcji. Przetestowany graf pokazano na rysunku 10. Test ten nie jest wykonywalny przez zaimplementowaną wykonalna aplikację.



Rysunek 10: Graf utworzony do przetestowania `dijkstra_stop`

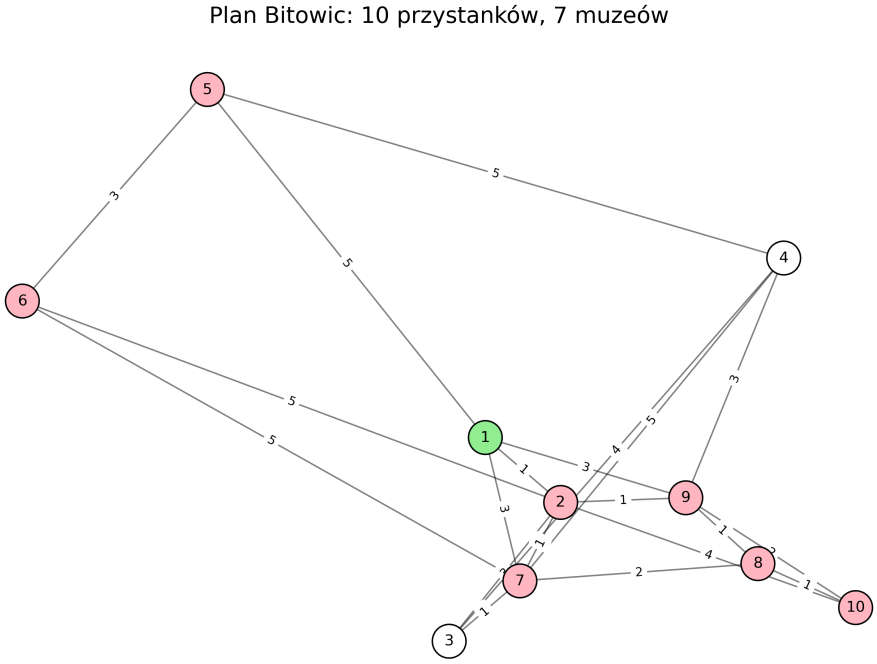
Graf ten jest zadany w postaci słownika list sąsiedztwa. Celem obliczeń było wyznaczenie najkrótszej ścieżki z wierzchołka A do wierzchołka D. Algorytm Dijkstry, zastosowany do powyższego grafu, zwrócił wartość:

$(7,\ [A, B, D])$,, co oznacza, że minimalny koszt przejścia wynosi 7, a ścieżka optymalna prowadzi przez wierzchołek B. Przykład ten potwierdza, że implementacja prawidłowo wyznacza zarówno minimalny dystans, jak i odpowiadającą mu ścieżkę.

Po zweryfikowaniu poprawności działania funkcji `dijkstra_stop` na przykładach elementarnych przeprowadzono test głównej implementacji, obejmującej pełną logikę zadania. Testy te wykonano już w aplikacji. Po wywołaniu generowane były przykładowe instancje problemu oraz przeprowadzono pełna symulację wycieczki Bajtazara. Program zwraca zarówno całkowity minimalny czas przejazdu, jak i wszystkie cząstkowe ścieżki pomiędzy kolejnymi parami muzeów. Na zakończenie działania funkcja przedstawia graficzną reprezentację użytego grafu, co umożliwia ręczną weryfikację spójności struktury oraz poprawności rozmieszczenia krawędzi i wag. Przeprowadzone testy potwierdziły, że implementacja poprawnie integruje generowanie danych, wyznaczanie najkrótszych ścieżek oraz końcową prezentację wyników, zgodnie z założeniami projektu. Testy zostały przeprowadzone dla każdej z trzech opcji wprowadzenia danych.

Poniżej omówiono wybrane przykłady wywołania implementacji, każdą z nich otrzymano po wybraniu opcji 2 w konsoli. A zatem we wszystkich z przypadków podawano liczby naturalne N, M oraz K w odpowiednich zakresach.

- `N=10`, `M=20` oraz `K=7`, wizualizacja została pokazana na rysunku 11.



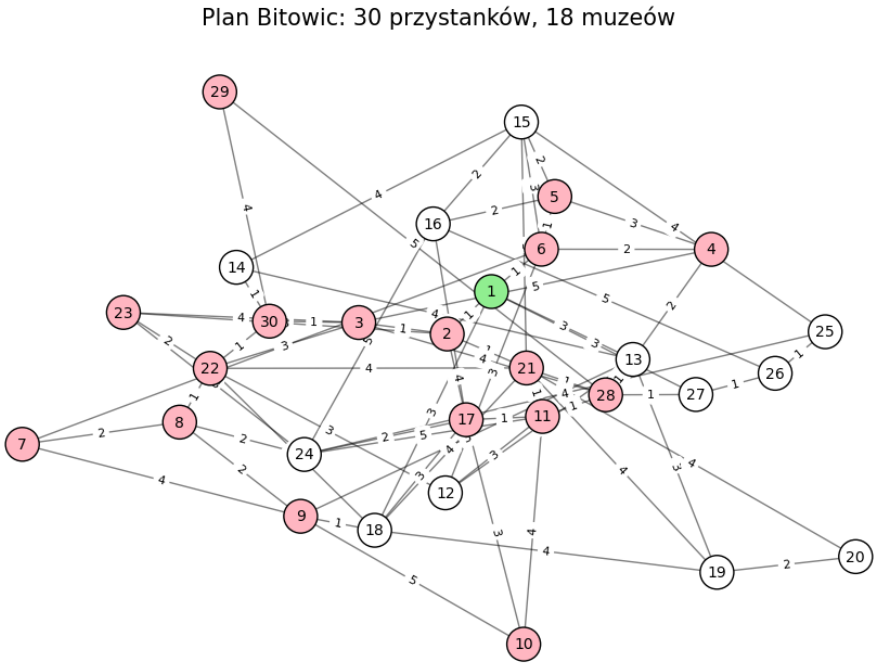
Rysunek 11: Ilustracja problemu wykonana dla pierwszego zestawu parametrów.

Powyższy przykład został celowo dobrany w taki sposób, aby zachować pełną prostotę i czytelność struktury grafu. Dzięki niewielkiej liczbie wierzchołków oraz przejrzystemu układowi połączeń możliwa jest łatwa, ręczna weryfikacja poprawności kodu. Tego typu przykład pozwala jednoznacznie prześledzić kolejne kroki algorytmu oraz upewnić się, że implementacja zwraca prawidłowe wartości. Stanowi on zatem istotny element walidacji wstępnej, zanim funkcja zostanie wykorzystana w bardziej złożonych scenariuszach. W tabeli 1 zwizualizowano wyniki otrzymane z konsoli.

Tabela 1: Wyniki przejazdów w wygenerowanym grafie (10 węzłów, 20 krawędzi, 7 muzeów).

Przejazd	**Czas**	**Ścieżka**
1 → 2	1	['1', '2']
2 → 5	6	['2', '1', '5']
5 → 6	3	['5', '6']
6 → 7	5	['6', '7']
7 → 8	2	['7', '8']
8 → 9	1	['8', '9']
9 → 10	2	['9', '10']
10 → 1	4	['10', '9', '2', '1']
Całkowity czas przejazdu: 24		
Pełna ścieżka: ['1', '2', '1', '5', '6', '7', '8', '9', '10', '9', '2', '1']		

- `N=30`, `M=50` oraz `K=18` W tym przypadku przeprowadzono analizę działania algorytmu na bardziej złożonym przykładzie, czego wizualizacja widoczna jest na rysunku 12. Przykład obejmuje większą liczbę wierzchołków, krawędzi oraz muzeów do odwiedzenia. Złożona struktura grafu zwiększa liczbę możliwych ścieżek alternatywnych, co umożliwia sprawdzenie, czy algorytm prawidłowo wybiera trasy minimalne nawet w sytuacji, gdy różnice pomiędzy potencjalnymi kosztami są niewielkie. Analiza pokazała, że implementacja poprawnie radzi sobie z przetwarzaniem większej liczby połączeń, zwracając nam tabelę wyników zwizualizowaną w tabeli 2. Program zachowuje spójność wyników oraz przewidywalność działania przy kolejnych iteracjach. Dzięki temu można uznać, że funkcja działa poprawnie również w warunkach zwiększonej złożoności strukturalnej.

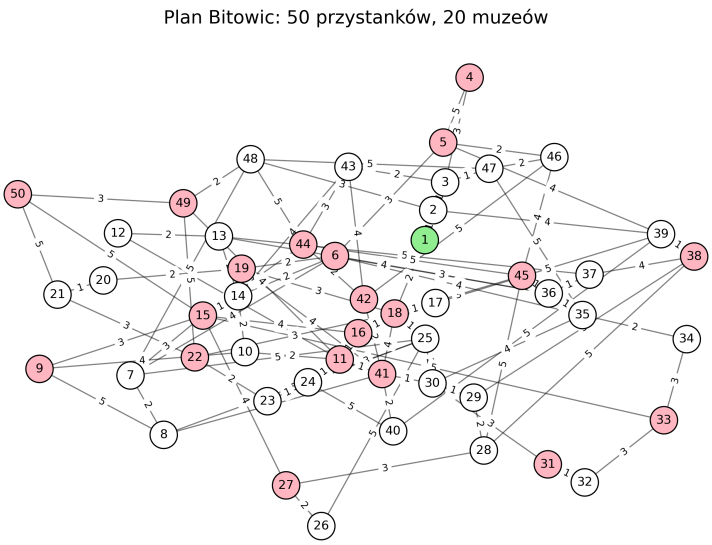


Rysunek 12: Ilustracja problemu wykonana dla drugiego zestawu parametrów.

Tabela 2: Wyniki przejazdów w wygenerowanym grafie (30 węzłów, 50 krawędzi, 18 muzeów).

Przejazd	**Czas**	**Ścieżka**
1 → 4	3	['1', '24', '4']
4 → 5	5	['4', '5']
5 → 6	4	['5', '6']
6 → 7	1	['6', '7']
7 → 8	1	['7', '8']
8 → 9	3	['8', '9']
9 → 10	3	['9', '10']
10 → 11	1	['10', '11']
11 → 12	1	['11', '12']
12 → 13	2	['12', '13']
13 → 14	3	['13', '14']
14 → 16	5	['14', '15', '16']
16 → 18	7	['16', '17', '18']
18 → 19	2	['18', '19']
19 → 21	5	['19', '20', '21']
21 → 22	3	['21', '22']
22 → 23	4	['22', '21', '23']
23 → 26	7	['23', '14', '2', '1', '25', '26']
26 → 1	3	['26', '25', '1']
Całkowity czas przejazdu: 63 **Pełna ścieżka:** ['1', '24', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16', '17', '18', '19', '20', '21', '22', '21', '23', '14', '2', '1', '25', '26', '25', '1']		

- `N=50`, `M=100` oraz `K=20`



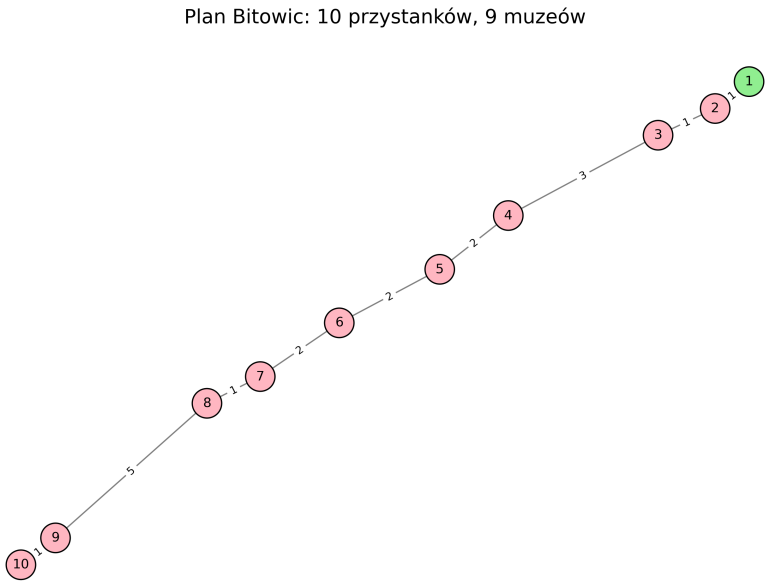
Rysunek 13: Ilustracja problemu wykonana dla trzeciego zestawu parametrów.

W trzecim teście wykorzystano maksymalne dopuszczalne parametry wejściowe, tj. $N = 100$, $M = 50$ oraz $K = 20$. Celem było sprawdzenie, czy implementacja poprawnie reaguje na dane znajdujące się na granicy specyfikacji oraz czy zachowuje stabilność działania w warunkach zwiększonej złożoności struktury grafu. W przypadku przekroczenia któregokolwiek z tych parametrów funkcja zgodnie ze specyfikacją zwraca odpowiedni komunikat błędu poprzez mechanizm wyjątków, chroniąc program przed przetwarzaniem niepoprawnych danych. Test wykazał, że mimo bardziej skomplikowanej struktury połączeń, generowany graf pozostaje spójny, co widać na rysunku 13, a algorytm `dijkstra_stop` poprawnie wyznacza kolejne minimalne ścieżki, czego potwierdzenie znajdujemy w tabeli 3. Ostateczne wyniki potwierdzają, że nawet w przypadku maksymalnych wartości parametrów problem zostaje rozwiązany zgodnie z założeniami, a implementacja zachowuje zarówno poprawność, jak i przewidywalność działania.

Tabela 3: Wyniki przejazdów w wygenerowanym grafie (50 węzłów, 100 krawędzi, 20 muzeów).

Przejazd	**Czas**	**Ścieżka**
1 → 4	5	['1', '2', '3', '4']
4 → 5	5	['4', '5']
5 → 6	3	['5', '6']
6 → 9	6	['6', '14', '15', '9']
9 → 11	6	['9', '10', '11']
11 → 15	5	['11', '16', '15']
15 → 16	4	['15', '16']
16 → 18	1	['16', '18']
18 → 19	3	['18', '19']
19 → 22	6	['19', '18', '22']
22 → 27	11	['22', '18', '25', '26', '27']
27 → 31	9	['27', '28', '29', '30', '31']
31 → 33	4	['31', '32', '33']
33 → 38	11	['33', '34', '35', '36', '37', '38']
38 → 41	7	['38', '29', '30', '41']
41 → 42	2	['41', '42']
42 → 44	2	['42', '44']
44 → 45	5	['44', '45']
45 → 49	9	['45', '18', '16', '11', '49']
49 → 50	3	['49', '50']
50 → 1	9	['50', '49', '48', '2', '1']
Całkowity czas przejazdu: 116		
Pełna ścieżka: ['1', '2', '3', '4', '5', '6', '14', '15', '9', '10', '11', '16', '15', '16', '18', '19', '18', '22', '18', '25', '26', '27', '28', '29', '30', '31', '32', '33', '34', '35', '36', '37', '38', '29', '30', '41', '42', '44', '45', '18', '16', '11', '49', '50', '49', '48', '2', '1']		

- `N=10`, `M=9` oraz `K=9`



Rysunek 14: Ilustracja problemu wykonana dla czwartego zestawu parametrów.

Ostatni test tej części programu przeprowadzono na grafie o bardzo specyficznej, liniowej strukturze, w tym scenariuszu liczba krawędzi jest minimalna niezbędna do zachowania spójności grafu ($M = N - 1$), co w efekcie tworzy strukturę łańcuchową. Taka konfiguracja wymusza na Bajtazarze odwiedzanie kolejnych punktów w sposób sekwencyjny, bez możliwości wyboru alternatywnych ścieżek czy skrótów. Wizualizację tej sytuacji przedstawiono na rysunku 14. Jak widać w tabeli 4, algorytm poprawnie zidentyfikował jedyną możliwą trasę przejazdu w jedną stronę (od 1 do 10), a następnie wyznaczył powrót tą samą drogą. Ten przypadek brzegowy potwierdza, że implementacja działa poprawnie również w sytuacjach skrajnych.

Tabela 4: Wyniki przejazdów w wygenerowanym grafie (10 węzłów, 9 krawędzi, 9 muzeów).

Przejazd	**Czas**	**Ścieżka**
$1 \to 2$	1	['1', '2']
$2 \to 3$	1	['2', '3']
$3 \to 4$	3	['3', '4']
$4 \to 5$	2	['4', '5']
$5 \to 6$	2	['5', '6']
$6 \to 7$	2	['6', '7']
$7 \to 8$	1	['7', '8']
$8 \to 9$	5	['8', '9']
$9 \to 10$	1	['9', '10']
$10 \to 1$	18	['10', '9', '8', '7', '6', '5', '4', '3', '2', '1']
Całkowity czas przejazdu: 36		
Pełna ścieżka: ['1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '9', '8', '7', '6', '5', '4', '3', '2', '1']		

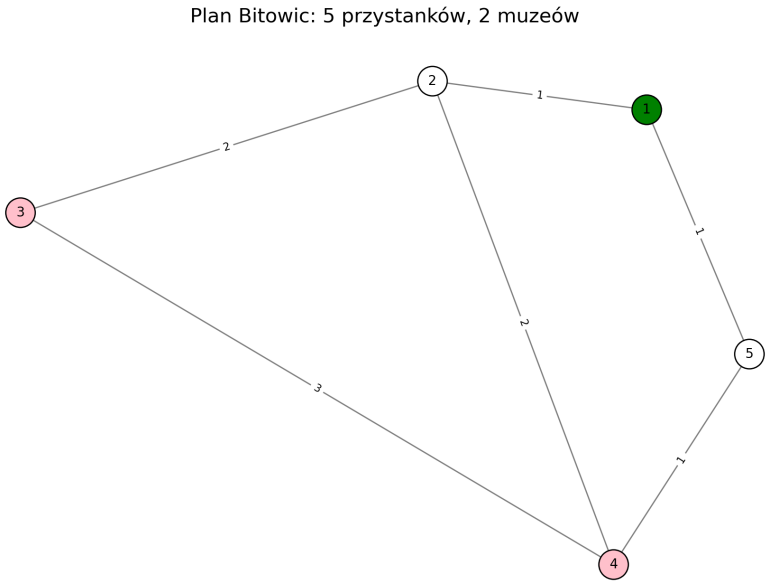
Drugą fazę testów przeprowadzono z wykorzystaniem opcji 1 menu głównego, czyli przy w całości ręcznym wprowadzaniu danych. Wybór ten podyktowany był koniecznością sprawdzenia poprawności algorytmu na w pełni kontrolowalnym zestawie danych, gdzie struktura grafu, wagi krawędzi oraz rozmieszczenie muzeów są precyzyjnie zdefiniowane przez użytkownika, a nie losowane. Pozwoliło to na porównanie wyniku zwróconego przez program z wynikiem obliczonym analitycznie. Problem zdefiniowany przez ręcznie podane parametry wraz z tymi parametrami przedstawiono na rysunku 15.

```
--- Tryb Ręczny ---
Podaj liczbę przystanków N (1-50): 5
Podaj liczbę połączeń M (min 4, max 100): 3
Błąd: Liczba musi być z zakresu od 4 do 100.
Podaj liczbę połączeń M (min 4, max 100): 6
Podaj liczbę muzeów K (1-5): 10
Błąd: Liczba musi być z zakresu od 1 do 5.
Podaj liczbę muzeów K (1-5): 2
Podaj numery 2 przystanków, gdzie są muzea:
Muzeum nr 1 (nr przystanku): 3
Muzeum nr 2 (nr przystanku): 3
To muzeum już zostało dodane!
Muzeum nr 2 (nr przystanku): 4

Definiowanie połączeń (format: x y t):
Gdzie: x, y - numery przystanków, t - czas przejazdu (max 5)
Krawędź 1/6: 1 2 1
Krawędź 2/6: 1 2 1
Błąd: Połączenie między 1 a 2 już istnieje.
Krawędź 2/6: 2 3 2
Krawędź 3/6: 3 4 8
Błąd: Czas przejazdu 't' musi być w zakresie 0-5.
Krawędź 3/6: 3 4 3
Krawędź 4/6: 232
Błąd: Wpisz dokładnie 3 liczby oddzielone spacją (x y t).
Krawędź 4/6: 2 4 2
Krawędź 5/6: 4 5 1
Krawędź 6/6: 5 1 1
----------------------------------------
```

Rysunek 15: Działanie mechanizmów walidacji danych wejściowych oraz obsługa błędów użytkownika (duplikaty, przekroczenie zakresów, błędny format).

Skonstruowany w ten sposób graf przedstawiono na rysunku 16, natomiast w tabeli 5



Rysunek 16: Ilustracja problemu wykonana dla danych wprowadzonych ręcznie.

Tabela 5: Wyniki testu ręcznego dla scenariusza N=5, M=6, K=2.

Przejazd	**Czas**	**Ścieżka**
$1 \to 3$	3	['1', '2', '3']
$3 \to 4$	3	['3', '4']
$4 \to 1$	2	['4', '5', '1']
Całkowity czas przejazdu: 8		
Pełna ścieżka: ['1', '2', '3', '4', '5', '1']		

Wynik działania programu dla tego testu był zgodny z oczekiwaniami. Aplikacja poprawnie zrekonstruowała zadaną strukturę i wyznaczyła optymalną ścieżkę o całkowitym koszcie wynoszącym 8. Potwierdza to poprawność mechanizmu wczytywania danych.

Wybór opcji 3 w konsoli programu od razu zwraca wynik wybrany losowo, zawsze zgodnie z założeniami zadania, bo tak są losowane te liczby. Wyniki losowej symulacji są takie same pod względem procedury jak wyniki w pozostałych przypadkach.

8 Wnioski końcowe

W ramach projektu zrealizowano kompletne rozwiązanie problemu optymalizacji trasy wycieczki w mieście Bitowice, wykorzystując algorytmy grafowe. Przeprowadzona analiza oraz testy pozwalają na sformułowanie następujących wniosków:

1. Zastosowanie zmodyfikowanego algorytmu Dijkstry z warunkiem wczesnego zatrzymania (Early Exit) okazało się kluczowym elementem optymalizacji. W porównaniu do klasycznej wersji algorytmu, która przeszukuje cały graf, nasze podejście znacząco redukuje liczbę operacji, zwłaszcza w przypadkach, gdy cel znajduje się blisko punktu startowego. Potwierdza to analiza złożoności obliczeniowej, która wykazała zależność czasu wykonania od faktycznej liczby odwiedzonych wierzchołków i^*, a nie tylko od rozmiaru całego grafu.

2. Przeprowadzone testy, obejmujące zarówno małe grafy weryfikowane ręcznie, jak i duże losowe struktury (do $N = 50$), wykazały pełną zgodność wyników z oczekiwaniami. Program radzi sobie poprawnie z różnymi topologiami sieci — od prostych łańcuchów po gęste grafy -– oraz z różnym zagęszczeniem muzeów.

3. Modułowa budowa programu (podział na walidację, generowanie grafu, algorytm wyszukiwania i logikę wycieczki) sprawia, że kod jest czytelny i łatwy w utrzymaniu. Zastosowanie reprezentacji grafu w postaci list sąsiedztwa zapewnia efektywność pamięciową i obliczeniową.

4. Dołączony moduł wizualizacji grafu okazał się nieocenionym narzędziem diagnostycznym. Pozwolił on na szybką ocenę poprawności generowanych struktur oraz na intuicyjne zrozumienie wyznaczanych przez algorytm ścieżek.

Podsumowując, cel projektu został w pełni osiągnięty. Stworzona implementacja skutecznie wyznacza minimalny czas przejazdu dla zadanego problemu, działając wydajnie w pełnym zakresie dopuszczalnych danych wejściowych. Projekt ten stanowi praktyczny przykład zastosowania teorii grafów do rozwiązywania rzeczywistych problemów logistycznych.

9 Podział prac

W tej sekcji przedstawimy dokładny podział nad projektem. Projekt w głównej mierze został zrealizowany wspólnie, a więc każdy z członków miał pełną kontrolę nad efektami pracy. Wszystkie postępy były zatem na bieżąco konsultowane oraz przedstawiana drugiej stronie.

Szczegółowy podział zadań:

1. Maria Eberhardt
   - Opis działania algorytmu Dijistry w raporcie.
   - Kompletne implementacja w języku Python części wizualizacyjnej grafów oraz algorytmów opisanych w pseudokodach.
   - Przeprowadzenie przykładowych wywołań wykonanej implementacji oraz uwzględnienie wyników w raporcie.
2. Krystian Kaliś
   - Przedstawienie oraz opis pseudokodów wykorzystanych algorytmów.
   - Analiza złożoności obliczeniowej zaimplementowanych algorytmów.
   - Implementacja struktury konsolowej programu, pliku .exe, zaimplementowanie funkcji potrzebnych do wywołania działania aplikacji.

Literatura

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2010. s. 658.

[2] DSA Dijkstra's Algorithm. `https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php`.

[3] Anna Maria Radzikowska. *Algorytmy i Struktury Danych* wykład 4: Algorytmy grafowe I. Wydział Matematyki i Nauk Informacyjnych, Politechnika Warszawska, 2025. Wykład akademicki slajd 25, `https://pages.mini.pw.edu.pl/~radzikowskaa/`.

[4] Jon Kleinberg and Eva Tardos. Algorithm design. *Pearson*, 2005. s. 290.